Matching the Reference to the Column

PA1: Lexing and Parsing ChocoPy

Mithulram Gunasekaran · Mohamed Nuhuman Mohamed Nafeel
Rajapaksha Mudiyanselage Dinodi Divyanjana Rajapaksha
Group 5 · Presentation 12.11.2026

Compiler Construction Lab · Winter Semester 2026/27
Chair of Software Engineering I · University of Passau

Outline

1. Motivation and task
2. Approach
3. Lexer
4. Parser
5. Error recovery
6. Evaluation
7. Conclusion and future work

Motivation: accepting ChocoPy is easy, matching it exactly is not

Valid program: the whole JSON tree must equal the reference's, including every `[line, col, line, col]` location.
Invalid program: the position of the first syntax error must match.

33/33	44/44	3,511	0	1.4 vs 2.0
provided sample tests	own edge-case tests	files identical to the reference	LALR conflicts	errors per real mistake (ours vs reference)

Our research question: which undocumented conventions does the reference follow — and can we recover from errors better than it does?

Approach: the reference implementation as an oracle

1 Read

Spec, manual §3–4, grammar figure, precedence table

2 Probe

Tiny inputs through the reference lexer and parser

3 Build

Lexer first (indentation), then the grammar

4 Fuzz

Random programs + mutants, compared by the grader's rule

5 Measure

A metric for recovery; keep only changes that help

THE PROBE THAT CRACKED THE LOCATION RULES

```
$ java ... chocopy.ChocoPy --pass=r coverage.py
IfStmt  [16, 13, 19, 8]     <- ends at line 19, column 8 ?!

19:         for x in xx:     <- line 19 is indented by 8 spaces
```

A block ends at the **indentation of the next line** — that is where its DEDENT is.

Outline

✓ 1. Motivation and task

✓ 2. Approach

3. Lexer

4. Parser

5. Error recovery

6. Evaluation

7. Conclusion and future work

Challenge 1: one line, many DEDENTs — one token per action

A JFlex action can return only one symbol. Line 6 below needs two DEDENTs.

```
3 |     for x in xs:
4 |         if x > 0:
5 |             s = s + x
6 |     return s
```

```
line:6 col:1 ----DEDENT--
line:6 col:1 ----DEDENT--
```

Why not a queue? It needs a wrapper around the generated next_token() and must stay in sync with JFlex's line/column counters. Re-scanning reuses JFlex itself.

OUR LOOP — NO TOKEN QUEUE NEEDED

A **Match leading whitespace + first char**
yypushback(1) the char

B **width < top of stack?**
pop one level, build one DEDENT

C **yypushback(yylength())**
next call re-scans the same whitespace

…until the stack and the line agree. The only state is the indentation stack.

Challenge 2: where exactly is a DEDENT?

Compound statements end at their closing DEDENT, so its position must match the reference exactly.

RULE WE REVERSE-ENGINEERED

```
DEDENT spans the NEXT line's leading
whitespace
  [L, 1, L, width]

  next line indented 8  ->  [19, 1, 19, 8]
  next line at column 1 ->  [5, 1, 5, 0]
```

Build the token before yypushback — afterwards yylength() is already 0.

END OF FILE: A REFERENCE QUIRK

Three blocks closing at EOF — the reference places their DEDENTs at

(5,1) (5,2) (5,3)

Columns that don't exist: outer blocks end one column after inner ones.

We mirror it (one commented line) and reported it as a bug. Missing final newline: behave as if "\n" were appended.

Challenge 3: strings — a build error, a redesign, two quirks

Each step was forced by evidence: first the compiler, then the differential tester.

v1 · build fails

```
[\u0020\u0021\u0023-
\u005B…]
```

JFlex expands \uXXXX before parsing the regex — \u005B became a bare [and broke the class. Fix: [!#-\[\]-~]

v2 · redesign

```
STRING state  →  one regex
+ stringSymbol() decoder
```

The reference reports an unterminated string as a single " token and keeps lexing. One whole-literal regex can do that.

v3 · quirks

```
"Fo\no"  ->  ONE token
[18,9,18,14]
```

The reference's match runs across lines to the next quote; a backslash before a line break ends it. Found by the fuzzer, mirrored.

Only \" \n \t \\ are legal. Any other escape, or a char outside ASCII 32–126, makes the whole literal one UNRECOGNIZED token.

Outline

✓ 1. Motivation and task

✓ 2. Approach

✓ 3. Lexer

4. Parser

5. Error recovery

6. Evaluation

7. Conclusion and future work

Grammar: the expr/cexpr split + CUP precedence

PRECEDENCE, LOW → HIGH

`· if · else ·`	right
`or`	left
`and`	left
`not`	unary
`== != < > <= >= is`	nonassoc
`+ -`	left
`* // %`	left
`- (unary, %prec UMINUS)`	unary
`. [ ]`	left

```
True == not False   →   syntax error
```

== joins two cexprs; not … is an expr. Exactly what the manual demands.

```
a < b < c   →   syntax error
```

Comparisons are nonassoc: a tie becomes an error entry in the table.

```
-a * b   =   (-a) * b,    -a.b   =   -(a.b)
```

Dummy terminal UMINUS gives unary minus its own level.

```
x : int    vs    x = 1
```

Same first token — LALR(1) decides on the next one (: or =).

0 conflicts — pom.xml has no expect, so one conflict would fail the build.

Challenge 4: locations come from grammar symbols, not AST nodes

Found by the fuzzer after we already passed all 33 samples.

Program starts with pass

```
pass
x = 1
```

Reference: Program starts at line 1.
We said line 2 — pass builds no node.

else: pass

```
if a:
    x = 1
else:
    pass
```

The list is empty, yet the IfStmt still ends at the else-block's DEDENT.

The coverage.py puzzle

method baz `[9,5,34,22]`
class bar `[6,1,36,0]`

The FuncDef node ends at its body; the func_def symbol includes the DEDENT of line 36 — the class ends at that symbol.

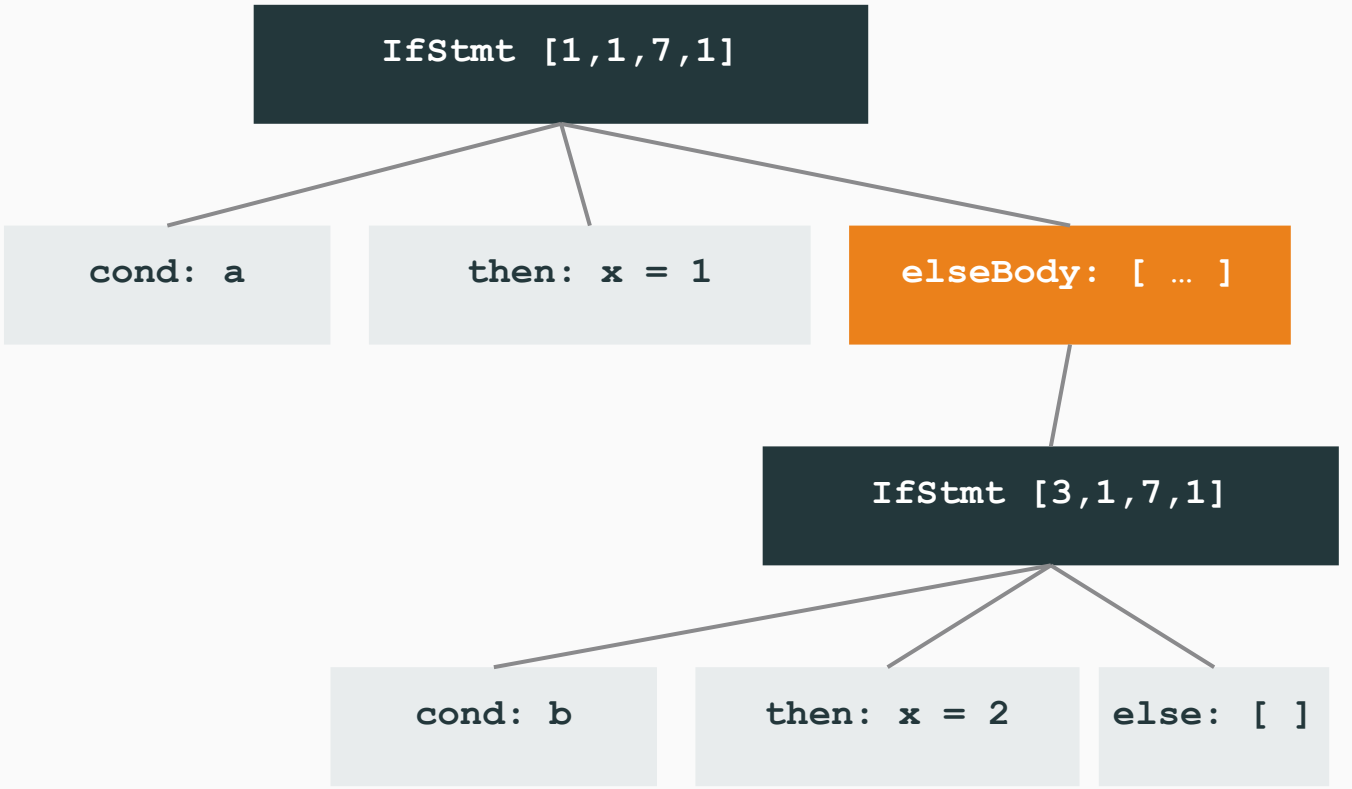
Fix: split the rules (if with / without else part; program with / without statements) so every location is `restxright, sxleft…` — never "is the list empty?"

Challenge 5: elif is sugar — one right-recursive rule

```
if a:
    x = 1
elif b:
    x = 2
else:
    pass
```

```
elif_else ::= ELSE COLON block
  | elif_header block
  | elif_header block elif_else

// each elif returns a one-element
// list holding a nested IfStmt
```



Both IfStmts end at (7,1): the DEDENT after the else-block — even though else: pass is an empty list.

Outline

✓ 1. Motivation and task

✓ 2. Approach

✓ 3. Lexer

✓ 4. Parser

5. Error recovery

6. Evaluation

7. Conclusion and future work

Challenge 6: one mistake, many errors — fixing the cascades

Three real mistakes in this file. How many errors does each parser report?

```
 1 | x:int = 1 + 2
 2 | y:int = 3
 4 | def broken(a, b:int) -> int:
 5 |     return b
 7 | def fine(n:int) -> int:
 8 |     return n
10 | z = 5
11 | lambda = 5
12 | print(fine(z))
```

Reference: 8

```
1:11 PLUS
2:2  COLON
4:1  DEF
4:16 COLON
7:1  DEDENT
7:11 COLON
10:1 DEDENT
11:1 RESERVED
```

Ours: 3

```
1:11 PLUS
4:13 COMMA
11:1 RESERVED
```

Recovery resumed on the leftover "2" as a statement — and declarations may not follow statements.

Anchored rules: `typed_var EQ error NEWLINE` · broken def/class/if/elif/while/for header → skip to its NEWLINE, then parse the optional indented body

Challenge 7: the vanishing statement

In the same file, line 10 is perfectly valid — and the reference drops it from the AST.

```
10 | z = 5
11 | lambda = 5
```

WHAT THE LR PARSER SEES

z	=	5	NEWLINE

← lookahead: RESERVED

Reducing stmt ::= simple_stmt NEWLINE needs a valid lookahead. RESERVED isn't one → error first → recovery pops the finished z = 5.

Fix: default reductions

- Consistent state (only one possible action: a reduction) → reduce on any lookahead, as Bison does by default.
- CUP has it as -compact_red, but that lives in pom.xml, which we may not edit → we patch CUP's action table in init with.
- Cannot move the first error: reductions consume no input.

```
ours       statements on lines [10, 11, 12]
reference  statements on lines [1, 5, 8, 11, 12]
```

Challenge 8: the abort trap — an anchor that never comes

After adding default reductions, one test produced an empty AST: CUP had given up on the whole file.

Anchored on the colon

```
IF error COLON block
```

1.30 errors / mistake
but 4 of 133 parses aborted

Why it aborts

CUP skips tokens until the anchor parses. No colon (or an error after the block) → EOF → give up

Our first metric hid it: it counted errors, not aborts

Final design

anchor on NEWLINE · body optional · headers as own nonterminals · function bodies may start with error

1.37 errors / mistake
0 aborts

Why header nonterminals? Once if_header ::= IF expr COLON is reduced, its states leave the stack — a later error can't fall back into the header's recovery rule.

Outline

✓ 1. Motivation and task

✓ 2. Approach

✓ 3. Lexer

✓ 4. Parser

✓ 5. Error recovery

6. Evaluation

7. Conclusion and future work

Evaluation I: correctness by differential testing

Grading also uses private tests, so we tested against the reference itself, using the grader's own rule.

`fuzz.py`

random valid programs: tabs/spaces, CR/LF/CRLF, comments, no final newline

→

`mutants`

each with one random defect

→

`difftest.py`

valid → full AST equal; invalid → first error equal

13

differences found after all 33 samples passed

3,511

files in the final run — all identical

193 / 400

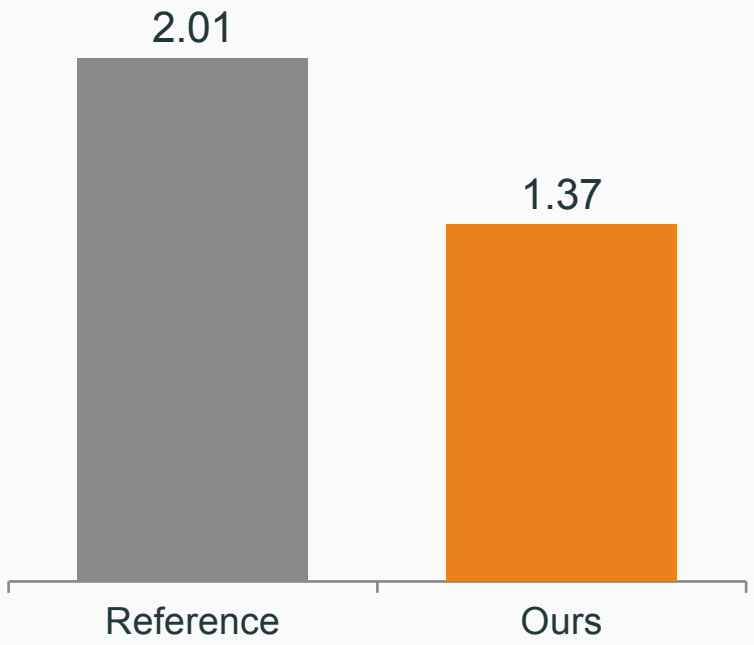
first generator: "valid" programs that really were — test the tester!

The generator built targets like -x.foo = 1 and chained comparisons. Fixed → 500/500 valid.

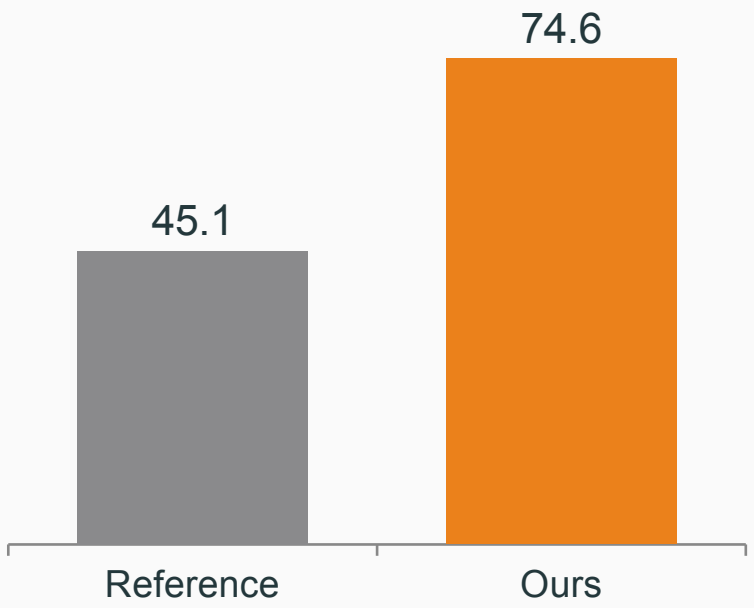
Evaluation II: recovery quality, one real mistake per program

tools/recovery_quality.py inserts one $ into a random line of 346 valid programs (two independent corpora).

Errors reported per real mistake



Exactly one error reported (%)



3 → 0

parses that gave up (reference → ours)

Also decided by data:
CUP's error_sync_size 3 → 2 → 1 made cascades worse (2.86 → 2.93 → 3.47). We kept 3.

Contribution: three bugs found in the reference

1 Unbalanced INDENT/DEDENT

```
if a:
    if b:
        c = 1
  d = 2
```

Pops only one level, then pushes 2 on top of 4 → 3 INDENTs, 2 DEDENTs → bogus "EOF" error. We pop all levels: one error.

2 Made-up EOF columns

```
if a:
  if b:
    if c:
      d
```

DEDENTs at (5,1) (5,2) (5,3): nested blocks end at columns that don't exist. Error text shows a stale "x".

3 Strings across lines

```
x = "abc
...
y = "d"
```

An unclosed quote swallows lines up to the next quote; end location [18,9,18,14] for a token ending on line 19.

All three are in the README with input, actual and expected output. We mirror #2 and #3 because locations are graded.

Outline

✓ 1. Motivation and task

✓ 2. Approach

✓ 3. Lexer

✓ 4. Parser

✓ 5. Error recovery

✓ 6. Evaluation

7. Conclusion and future work

Limitations and future work

Open problems

- A statement with a broken header is dropped: its body is parsed and checked, but not kept in the AST.
- A broken declaration inside a function (not an initial value) switches recovery to statement mode.
- An error directly after an error can go unreported (CUP's 3-token window).
- Two reference quirks mirrored on purpose — one line each to undo.

Future work (PA2 and beyond)

- Friendlier messages, e.g. "unindent does not match any outer level" instead of UNRECOGNIZED.
- Placeholder nodes so broken statements keep their body — only if PA2 can ignore them safely.
- Reuse difftest/fuzz/recovery_quality for semantic analysis (PA2) and code generation (PA3).

Conclusion

1. **Correct** — Identical to the reference on 3,511 files, 0 conflicts, all samples and edge cases pass.

2. **Better recovery** — 1.37 vs 2.01 errors per mistake, 0 vs 3 aborted parses — measured, not guessed.

3. **Evidence-driven** — Oracle probing + differential fuzzing found every hard bug; the samples found none.

4. **Lessons** — Decide node vs symbol locations once · build the tester on day one · measure the failure you fear most.

Thank you — questions?

`Live demo:  bash demo/demo.sh`

Mithulram Gunasekaran · Mohamed Nuhuman Mohamed Nafeel
Rajapaksha Mudiyanselage Dinodi Divyanjana Rajapaksha
Group 5 · Compiler Construction Lab WS 2026/27 · Chair of Software Engineering I · University of Passau

Backup: how CUP recovers from an error

1. Report the error (syntax_error)
2. Pop states until one can shift the special error token
3. Shift error (then reduce, e.g. stmt_list ::= stmt_list error)
4. Discard input until the next 3 tokens (error_sync_size) parse
5. Continue — or reach EOF first and give up entirely

Why separate lists?

After the declarations, a declaration or a statement may begin. Two live error rules there (decls ::= decls error and stmts ::= error) end in one state with two completed items → reduce/reduce conflict. So top-level statements accept error only after the first statement; blocks and function bodies may start with it.

Backup: running it

```
cd chocopy-pa1 && mvn clean package

# tests
java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.ChocoPy --pass=s \
     --dir src/test/data/pa1/sample --test

# one file: ours (s) / reference (r)
java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.ChocoPy --pass=s file.py

# tokens (INDENT/DEDENT)
java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.pa1.ChocoPyLexer file.py

# differential testing
python3 tools/fuzz.py /tmp/fuzz 200 42 && python3 tools/difftest.py /tmp/fuzz
```